DD2482 - 10/10/26

Project Report

Alexander Wigren & Dawa Arkhang

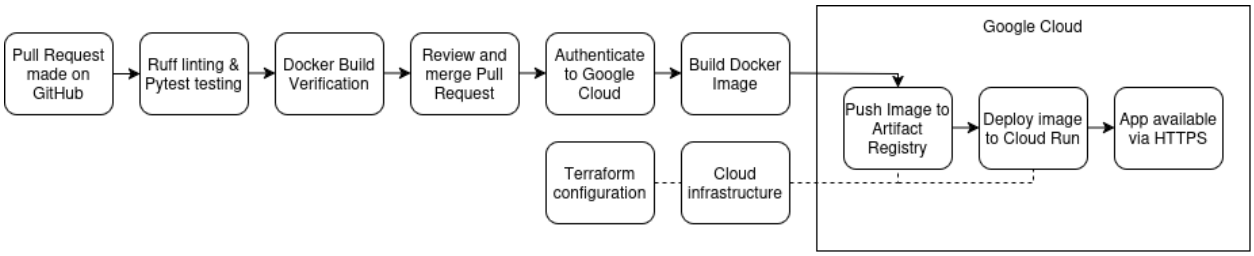
Introduction

Our project consists of a small, open-source CRUD (Create, Remove, Update and Delete) application, originally created by Samim Sarkar, which we have modified. We have then taken this small app and built a CI/CD pipeline around this app in accordance with DevOps practices that are taught in the course. In the sections below we give more detailed descriptions of our architecture and the interactions between tools, elaborate on the design decisions we made and also discuss limitations and trade-offs.

Architecture

Application: The app we chose to build our project around is a very simple CRUD implementation built in Python with a SQLit3 database.
Version Control & Development platform: We use Git for version control and GitHub as our collaboration and development platform.
CI: For our CI (Continuous Integration) pipeline we use Pytest to test our code, the Ruff linter to check our code for formatting errors and simple bugs, and Docker to containerize the application. These all run as sequential jobs implemented with GitHub Actions.
CD: Once the testing in the previous steps succeeds, our automated CD (Continuous Deployment) workflow builds a docker image and pushes it to Google Artifact Registry. It then automatically deploys a new revision(containing the specific Docker image, environment settings, and port configuration that were deployed together) to Google Cloud Run. This exposes the service publicly over HTTPS. Authentication between GitHub actions and Google Cloud is done keyless by using Workload Identity Federation (WIF) via OpenID Connect (OIDC).
IaC: We use Terraform as our IaC (Infrastructure as Code) solution to handle the provisioning and configuration of our chosen cloud platform.

Cloud Platform: We host the containerized application on Google Cloud Platform (GCP) in the europe-north1 (Finland) region, using Google Cloud Run for serverless container execution and Google Artifact Registry for private container image storage.



Design Decisions

We chose the application we built the CI/CD pipeline around primarily for three reasons:

• It is free and open-source

• It is small and and easily understood, yet is not a trivial hello-world style program

• It did not already have any CI/CD/DevOps elements added already which we would have needed to remove before starting to work on it ourselves.

Regarding our choices of development platform, testing framework and linter, these all came as rather natural consequences of our chosen app and previous experiences. Pytest is probably the most widely used testing framework for Python software and has a myriad of resources and tutorials available. Likewise, Ruff is a very well known linter specifically for the Python language and provides excellent utility as a quality assurance tool while simultaneously being quite simple to set up. Regarding GitHub and GitHub Actions, we both already had accounts on the platform and were familiar with its use, and the course itself also has its main pages there, leading to it becoming the obvious choice for our purposes.

We used Docker to ensure the flask app runs identically both locally and in production. During deployment, the workflow builds a Docker image and tags it with both the git commit hash and the latest. Tagging with the commit hash makes tracking which version is currently live and rolling back changes straightforward when needed.

We chose to deploy our app on Google Cloud Run due to its handy container management, automatic HTTPS and scaling without needing a dedicated vm. Instead of using Docker Hub

we push our images directly to google artifact registry which keeps the container images private to our GCP project and speeds up deployment time.

To connect GitHub Actions to Google Cloud securely, we chose workload identity federation(WIF) instead of downloading and storing static jason service account keys inGitHub secrets. This allows GitHub Actions to authenticate using short lived tokens which then avoids leaking permanent keys.

Limitations and Trade-offs

• Since our chosen application is quite small and simplistic, it may not accurately reflect how a complete CI/CD environment looks in a typical production setting. We feel like this is a reasonable concession given the limited time and scope of the project given by the grading criteria.
• Our implementation of automated testing/linting may help improve code quality, but it is by no means a guarantee that the application is bug-free or completely secure.
• The application only runs a local SQLit3 database server and therefore data may be lost upon a Docker image being torn down and replaced with a new, updated version. We argue that this is a reasonable limitation as the application itself merely serves a secondary role in this course, while the focus is on the surrounding DevOps practices and CI/CD pipeline. Setting up a persistent, remote database server would not provide any utility from the perspective of the course learning goals.

A note on our usage of AI

While building the project we have made use of artificial intelligence tools. These include:

• Bouncing around ideas with various chatbots like ChatGPT, Claude and Gemini

• Using GitHub Copilot when writing and editing code/configuration files

While using these tools we have taken care to manually review and verify the output as correct, and when necessary perform manual editing. We would like to clarify that this report itself was *not* made with AI.